VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

PRACTICAL PROJECT

DATA STRUCTURES & ALGORITHMS

Course: Data Structures & Algorithms
Course Code: CSC10004
Semester: Semester 2 – 2025/2026

Instructor Information

Instructor: Huynh Lam Hai Dang, MSc
Le Nhut Nam, MSc
E-mail: {hlhdang, lnnam}@fit.hcmus.edu.vn]
Department: Computer Science

Contents

1 Project Overview **2**

2 Project Requirements **2**

2.1 Team Composition 2

2.2 Written Report 2

2.3 Demo Implementation (C/C++) 3

2.4 Comparative Study 4

2.5 Optional: Improvements 4

3 Grading Criteria **4**

4 Deliverables **5**

1. Project Overview

This practical project is designed for students enrolled in the **Data Structures & Algorithms** course. Each group will investigate a specific algorithm or data structure assigned by the instructor, implement a working demo in C/C++, and produce a written report documenting their findings.

Requirements

Each group consists of **exactly 4 students**. The topic (an algorithm or data structure) is **assigned by the instructor** and may not be changed without prior approval.

2. Project Requirements

2.1. Team Composition

- Groups must consist of **4 students**.
- Each group is assigned **one topic** by the instructor, covering either a specific algorithm or a data structure.
- All members are expected to contribute equally to both the report and the implementation.

2.2. Written Report

The group must produce a written report (in **Vietnamese or English**) covering the following:

Report Contents

1. **Introduction** – Motivation and relevance of the topic.
2. **Theoretical Background** – Formal definition, properties, and key concepts of the assigned algorithm or data structure.
3. **Algorithmic Analysis** – Time and space complexity analysis using Big-O notation: $\mathcal{O}(n)$, $\mathcal{O}(n \log n)$, etc.
4. **Implementation Details** – Description of the C/C++ implementation and design decisions.
5. **Comparative Study** – Comparison with related algorithms or data structures in terms of performance and use cases.

6. **Improvements (if any)** – Any optimizations or enhancements proposed or implemented by the group.
7. **Conclusion** – Summary of findings and lessons learned.

Note

The report must be submitted as a **PDF file**. It should be well-structured, clearly written, and include figures or tables where appropriate to support the analysis.

2.3. Demo Implementation (C/C++)

Each group must implement a working demo program in **C or C++** that:

- Demonstrates the **core operations** of the assigned algorithm or data structure (e.g., insertion, deletion, search, traversal).
- Is **well-commented** and readable.
- Includes **test cases** that validate correctness on various inputs (edge cases, large inputs, etc.).
- Measures and reports **execution time** to support performance evaluation.

A minimal code structure is suggested below:

```cpp
#include <iostream>
#include <chrono>
using namespace std;
using namespace std::chrono;

// --- Data structure / Algorithm implementation ---
// TODO: Implement your assigned topic here

// --- Performance measurement helper ---
void measureTime(/* parameters */) {
    auto start = high_resolution_clock::now();
    // call your function
    auto end   = high_resolution_clock::now();
    duration<double, milli> elapsed = end - start;
    cout << "Elapsed time: " << elapsed.count() << " ms\n";
}

int main() {
    // TODO: Run demo and test cases
    return 0;
}
```

Listing 1: Suggested structure for demo program (C++)

2.4. Comparative Study

Groups must benchmark and compare their implementation against **at least one related algorithm or data structure**. The comparison should address:

- **Time complexity** – theoretical Big-O analysis.
- **Empirical performance** – measured running time on datasets of varying sizes.
- **Space usage** – memory overhead comparison.
- **Practical trade-offs** – scenarios where one approach is preferred over the other.

2.5. Optional: Improvements

Groups are encouraged (but not required) to propose or implement improvements to the base algorithm or data structure. This may include:

- Optimizing time or space complexity.
- Adapting the structure for a specific real-world application.
- Extending functionality beyond the standard definition.

3. Grading Criteria

The project will be evaluated based on the following rubric:

Criterion	Weight	Max Score
Written Report (clarity, depth, structure)	35%	3.5 pts
Demo Code (correctness, readability, tests)	35%	3.5 pts
Comparative Study (analysis & benchmarks)	20%	2.0 pts
Improvements / Bonus	10%	1.0 pt
Total	**100%**	**10 pts**

Note

Plagiarism of code or report content will result in a **score of 0** for all involved parties. All submissions are subject to similarity checks.

4. Deliverables

Deliverables

1. **PDF Report** – Written report following the structure outlined in Section 2.2.
2. **Source Code** – All C/C++ source files, organized in a clean folder structure with a `README.md` describing how to compile and run the demo.
3. **Submission format:** A single `.zip` archive named `GroupXX.zip` (e.g., `Group03.zip`).

Submission Checklist

- ☐ Report is complete and exported as PDF.
- ☐ All source code compiles without errors.
- ☐ Test cases are included and documented.
- ☐ Performance benchmarks are included in the report.
- ☐ Archive is named correctly before submission.